

OpenAI Function Calling In LangChain

```
In [1]: import os
import openai

from dotenv import load_dotenv, find_dotenv
_ = load_dotenv(find_dotenv()) # read local .env file
openai.api_key = os.environ['OPENAI_API_KEY']
```

```
In [2]: from typing import List
from pydantic import BaseModel, Field
```

Pydantic Syntax

Pydantic data classes are a blend of Python's data classes with the validation power of Pydantic.

They offer a concise way to define data structures while ensuring that the data adheres to specified types and constraints.

In standard python you would create a class like this:

```
In [3]: class User:
def __init__(self, name: str, age: int, email: str):
    self.name = name
    self.age = age
    self.email = email
```

```
In [4]: foo = User(name="Joe", age=32, email="joe@gmail.com")
```

```
In [5]: foo.name
```

```
Out[5]: 'Joe'
```

```
In [6]: foo = User(name="Joe", age="bar", email="joe@gmail.com")
```

```
In [7]: foo.age
```

```
Out[7]: 'bar'
```

```
In [8]: class pUser(BaseModel):
    name: str
    age: int
    email: str
```

```
In [9]: foo_p = pUser(name="Jane", age=32, email="jane@gmail.com")
```

```
In [10]: foo_p.name
```

```
Out[10]: 'Jane'
```

Note: The next cell is expected to fail.

```
In [ ]: foo_p = pUser(name="Jane", age="bar", email="jane@gmail.com")
```

```
In [12]: class Class(BaseModel):  
         students: List[pUser]
```

```
In [13]: obj = Class(  
         students=[pUser(name="Jane", age=32, email="jane@gmail.com")]  
         )
```

```
In [14]: obj
```

```
Out[14]: Class(students=[pUser(name='Jane', age=32, email='jane@gmail.com')])
```

Pydantic to OpenAI function definition

```
In [15]: class WeatherSearch(BaseModel):  
         """Call this with an airport code to get the weather at that airport"""  
         airport_code: str = Field(description="airport code to get weather for")
```

```
In [16]: from langchain.utils.openai_functions import convert_pydantic_to_openai_func
```

```
In [17]: weather_function = convert_pydantic_to_openai_function(WeatherSearch)
```

```
In [18]: weather_function
```

```
Out[18]: {'name': 'WeatherSearch',  
          'description': 'Call this with an airport code to get the weather at that  
airport',  
          'parameters': {'title': 'WeatherSearch',  
                          'description': 'Call this with an airport code to get the weather at that  
airport',  
                          'type': 'object',  
                          'properties': {'airport_code': {'title': 'Airport Code',  
                                                          'description': 'airport code to get weather for',  
                                                          'type': 'string'}}},  
          'required': ['airport_code']}
```

```
In [19]: class WeatherSearch1(BaseModel):  
         airport_code: str = Field(description="airport code to get weather for")
```

Note: The next cell is expected to generate an error.

```
In [ ]: convert_pydantic_to_openai_function(WeatherSearch1)
```

```

In [20]: class WeatherSearch2(BaseModel):
        """Call this with an airport code to get the weather at that airport"""
        airport_code: str

In [21]: convert_pydantic_to_openai_function(WeatherSearch2)

Out[21]: {'name': 'WeatherSearch2',
        'description': 'Call this with an airport code to get the weather at that airport',
        'parameters': {'title': 'WeatherSearch2',
        'description': 'Call this with an airport code to get the weather at that airport',
        'type': 'object',
        'properties': {'airport_code': {'title': 'Airport Code', 'type': 'string'}},
        'required': ['airport_code']}}

In [22]: from langchain.chat_models import ChatOpenAI

In [23]: model = ChatOpenAI()

In [24]: model.invoke("what is the weather in SF today?", functions=[weather_function])

Out[24]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Weather Search', 'arguments': '{"airport_code":"SF0"}'}})

In [25]: model_with_function = model.bind(functions=[weather_function])

In [26]: model_with_function.invoke("what is the weather in sf?")

Out[26]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Weather Search', 'arguments': '{"airport_code":"SF0"}'}})

```

Forcing it to use a function

We can force the model to use a function

```

In [27]: model_with_forced_function = model.bind(functions=[weather_function], function_call="call")

In [28]: model_with_forced_function.invoke("what is the weather in sf?")

Out[28]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Weather Search', 'arguments': '{"airport_code":"SF0"}'}})

In [29]: model_with_forced_function.invoke("hi!")

Out[29]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Weather Search', 'arguments': '{"airport_code":"SF0"}'}})

```

Using in a chain

We can use this model bound to function in a chain as we normally would

```
In [30]: from langchain.prompts import ChatPromptTemplate
```

```
In [31]: prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant"),
    ("user", "{input}")
])
```

```
In [32]: chain = prompt | model_with_function
```

```
In [33]: chain.invoke({"input": "what is the weather in sf?"})
```

```
Out[33]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Weather
Search', 'arguments': '{"airport_code": "SF0"}'}})
```

Using multiple functions

Even better, we can pass a set of function and let the LLM decide which to use based on the question context.

```
In [34]: class ArtistSearch(BaseModel):
    """Call this to get the names of songs by a particular artist"""
    artist_name: str = Field(description="name of artist to look up")
    n: int = Field(description="number of results")
```

```
In [35]: functions = [
    convert_pydantic_to_openai_function(WeatherSearch),
    convert_pydantic_to_openai_function(ArtistSearch),
]
```

```
In [36]: model_with_functions = model.bind(functions=functions)
```

```
In [37]: model_with_functions.invoke("what is the weather in sf?")
```

```
Out[37]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'Weather
Search', 'arguments': '{"airport_code": "SF0"}'}})
```

```
In [38]: model_with_functions.invoke("what are three songs by taylor swift?")
```

```
Out[38]: AIMessage(content='', additional_kwargs={'function_call': {'name': 'ArtistS
earch', 'arguments': '{"artist_name": "Taylor Swift", "n": 3}'}))
```

```
In [39]: model_with_functions.invoke("hi!")
```

```
Out[39]: AIMessage(content='Hello! How can I assist you today?')
```

```
In [ ]:
```

```
In [ ]:
```